

Window Management System - Representational System Solution

1. Ontology Definition

Our ontology consists of the following categories:

- **Situations:** Points in time representing the state of the world
- **Actions:** Operations that can be performed on windows
- **Windows:** Individual window objects in the interface
- **Integers:** For coordinates and ordering values
- **WindowStates:** Enumeration of possible window states
- **Positions:** Coordinate pairs for window locations
- **Dimensions:** Width and height values for window sizes

2. Language Definition

2.1 Constants

- **S_0 :** Initial situation
- **NIL:** Null/undefined value
- **MINIMIZED, DISPLAYED, NONEXISTENT:** Window state constants

2.2 Function Symbols

Situation Calculus Functions:

- **Do(a, s):** The situation resulting from performing action a in situation s
- **Result(a, s):** Synonym for Do(a, s)

Window Property Functions:

- **WindowState(w, s):** Returns the state of window w in situation s
- **ActiveWindow(s):** Returns the currently active window in situation s
- **WindowPosition(w, s):** Returns the (x,y) position of window w in situation s
- **WindowSize(w, s):** Returns the (width, height) of window w in situation s
- **WindowOrder(w, s):** Returns the z-order (depth) of window w in situation s

- **WindowExists(w, s)**: Boolean function indicating if window w exists in situation s

Utility Functions:

- **MakePosition(x, y)**: Creates a position object from coordinates
- **MakeDimension(w, h)**: Creates a dimension object from width and height
- **GetX(pos)**: Extracts x-coordinate from position
- **GetY(pos)**: Extracts y-coordinate from position
- **GetWidth(dim)**: Extracts width from dimension
- **GetHeight(dim)**: Extracts height from dimension

2.3 Predicate Symbols

State Predicates:

- **WindowHasState(w, state, s)**: Window w has the specified state in situation s
- **IsActive(w, s)**: Window w is the active window in situation s
- **IsDisplayed(w, s)**: Window w is currently displayed in situation s
- **IsMinimized(w, s)**: Window w is minimized in situation s
- **Exists(w, s)**: Window w exists in situation s
- **Overlaps(w1, w2, s)**: Windows w1 and w2 overlap in situation s
- **InFront(w1, w2, s)**: Window w1 is in front of window w2 in situation s

Action Predicates:

- **Poss(a, s)**: Action a is possible in situation s

2.4 Action Terms

- **CreateWindow(w, x, y, width, height)**: Create window w at position (x,y) with given dimensions
- **DestroyWindow(w)**: Destroy window w
- **MoveWindow(w, x, y)**: Move window w to position (x,y)
- **ResizeWindow(w, width, height)**: Resize window w to new dimensions
- **MinimizeWindow(w)**: Minimize window w
- **DisplayWindow(w)**: Display/restore window w
- **BringToFront(w)**: Bring window w to the front
- **ActivateWindow(w)**: Make window w the active window

3. Axioms

3.1 Effect Axioms

CreateWindow Effect:

```
WindowHasState(w, DISPLAYED, Do(CreateWindow(w, x, y, width, height), s)) ←  
.... ¬Exists(w, s)  
  
WindowPosition(w, Do(CreateWindow(w, x, y, width, height), s)) = MakePosition(x, y) ←  
.... ¬Exists(w, s)  
  
WindowSize(w, Do(CreateWindow(w, x, y, width, height), s)) = MakeDimension(width, height)  
←  
.... ¬Exists(w, s)  
  
Exists(w, Do(CreateWindow(w, x, y, width, height), s)) ←  
.... ¬Exists(w, s)
```

DestroyWindow Effect:

```
WindowHasState(w, NONEXISTENT, Do(DestroyWindow(w), s)) ←  
.... Exists(w, s)  
  
¬Exists(w, Do(DestroyWindow(w), s)) ←  
.... Exists(w, s)  
  
ActiveWindow(Do(DestroyWindow(w), s)) = NIL ←  
.... ActiveWindow(s) = w
```

MoveWindow Effect:

```
WindowPosition(w, Do(MoveWindow(w, x, y), s)) = MakePosition(x, y) ←  
.... Exists(w, s) ∧ WindowHasState(w, DISPLAYED, s)
```

ResizeWindow Effect:

```
WindowSize(w, Do(ResizeWindow(w, width, height), s)) = MakeDimension(width, height) ←  
.... Exists(w, s) ∧ WindowHasState(w, DISPLAYED, s)
```

MinimizeWindow Effect:

```

WindowHasState(w, MINIMIZED, Do(MinimizeWindow(w), s)) ←
.... Exists(w, s) ∧ WindowHasState(w, DISPLAYED, s)

ActiveWindow(Do(MinimizeWindow(w), s)) = NIL ←
.... ActiveWindow(s) = w

```

DisplayWindow Effect:

```

WindowHasState(w, DISPLAYED, Do(DisplayWindow(w), s)) ←
.... Exists(w, s) ∧ WindowHasState(w, MINIMIZED, s)

```

BringToFront Effect:

```

WindowOrder(w, Do(BringToFront(w), s)) = 0 ←
.... Exists(w, s) ∧ WindowHasState(w, DISPLAYED, s)

WindowOrder(w2, Do(BringToFront(w1), s)) = WindowOrder(w2, s) + 1 ←
.... w1 ≠ w2 ∧ Exists(w2, s) ∧ WindowHasState(w2, DISPLAYED, s) ∧
.... WindowOrder(w2, s) ≤ WindowOrder(w1, s)

```

ActivateWindow Effect:

```

ActiveWindow(Do(ActivateWindow(w), s)) = w ←
.... Exists(w, s) ∧ WindowHasState(w, DISPLAYED, s)

```

3.2 Frame Axioms

Window State Frame Axiom:

```

WindowHasState(w, state, Do(a, s)) ←
.... WindowHasState(w, state, s) ∧
.... ¬(∃x, y, width, height. a = CreateWindow(w, x, y, width, height)) ∧
.... a ≠ DestroyWindow(w) ∧
.... a ≠ MinimizeWindow(w) ∧
.... a ≠ DisplayWindow(w)

```

Window Position Frame Axiom:

```

WindowPosition(w, Do(a, s)) = WindowPosition(w, s)  $\leftarrow$ 
.....  $(\exists x, y. a = \text{MoveWindow}(w, x, y)) \vee$ 
.....  $(\exists x, y, \text{width}, \text{height}. a = \text{CreateWindow}(w, x, y, \text{width}, \text{height})) \vee$ 
.....  $a = \text{DestroyWindow}(w)$ 

```

Window Size Frame Axiom:

```

WindowSize(w, Do(a, s)) = WindowSize(w, s)  $\leftarrow$ 
.....  $\neg(\exists \text{width}, \text{height}. a = \text{ResizeWindow}(w, \text{width}, \text{height})) \wedge$ 
.....  $\neg(\exists x, y, \text{width}, \text{height}. a = \text{CreateWindow}(w, x, y, \text{width}, \text{height})) \wedge$ 
.....  $a \neq \text{DestroyWindow}(w)$ 

```

3.3 Precondition Axioms

Action Possibility Axioms:

```

Poss(CreateWindow(w, x, y, width, height), s)  $\leftrightarrow$ 
.....  $\neg \text{Exists}(w, s) \wedge x \geq 0 \wedge y \geq 0 \wedge \text{width} > 0 \wedge \text{height} > 0$ 

```

```

Poss(DestroyWindow(w), s)  $\leftrightarrow$ 
.....  $\text{Exists}(w, s)$ 

```

```

Poss(MoveWindow(w, x, y), s)  $\leftrightarrow$ 
.....  $\text{Exists}(w, s) \wedge \text{WindowHasState}(w, \text{DISPLAYED}, s) \wedge x \geq 0 \wedge y \geq 0$ 

```

```

Poss(ResizeWindow(w, width, height), s)  $\leftrightarrow$ 
.....  $\text{Exists}(w, s) \wedge \text{WindowHasState}(w, \text{DISPLAYED}, s) \wedge \text{width} > 0 \wedge \text{height} > 0$ 

```

```

Poss(MinimizeWindow(w), s)  $\leftrightarrow$ 
.....  $\text{Exists}(w, s) \wedge \text{WindowHasState}(w, \text{DISPLAYED}, s)$ 

```

```

Poss(DisplayWindow(w), s)  $\leftrightarrow$ 
.....  $\text{Exists}(w, s) \wedge \text{WindowHasState}(w, \text{MINIMIZED}, s)$ 

```

```

Poss(BringToFront(w), s)  $\leftrightarrow$ 
.....  $\text{Exists}(w, s) \wedge \text{WindowHasState}(w, \text{DISPLAYED}, s)$ 

```

```

Poss(ActivateWindow(w), s)  $\leftrightarrow$ 
.....  $\text{Exists}(w, s) \wedge \text{WindowHasState}(w, \text{DISPLAYED}, s)$ 

```

3.4 State Constraint Axioms

Unique Active Window:

$$\forall w_1, w_2, s. (\text{IsActive}(w_1, s) \wedge \text{IsActive}(w_2, s)) \rightarrow w_1 = w_2$$

Window State Constraints:

$$\begin{aligned} \forall w, s. \text{Exists}(w, s) \rightarrow \\ \dots (\text{WindowHasState}(w, \text{DISPLAYED}, s) \vee \\ \dots \text{WindowHasState}(w, \text{MINIMIZED}, s) \vee \\ \dots \text{WindowHasState}(w, \text{NONEXISTENT}, s)) \end{aligned}$$
$$\forall w, s. \neg \text{Exists}(w, s) \rightarrow \text{WindowHasState}(w, \text{NONEXISTENT}, s)$$

Overlap Definition:

$$\begin{aligned} \text{Overlaps}(w_1, w_2, s) \leftrightarrow \\ \dots w_1 \neq w_2 \wedge \\ \dots \text{WindowHasState}(w_1, \text{DISPLAYED}, s) \wedge \\ \dots \text{WindowHasState}(w_2, \text{DISPLAYED}, s) \wedge \\ \dots \text{GetX}(\text{WindowPosition}(w_1, s)) < \text{GetX}(\text{WindowPosition}(w_2, s)) + \text{GetWidth}(\text{WindowSize}(w_2, s)) \wedge \\ \dots \text{GetX}(\text{WindowPosition}(w_2, s)) < \text{GetX}(\text{WindowPosition}(w_1, s)) + \text{GetWidth}(\text{WindowSize}(w_1, s)) \wedge \\ \dots \text{GetY}(\text{WindowPosition}(w_1, s)) < \text{GetY}(\text{WindowPosition}(w_2, s)) + \text{GetHeight}(\text{WindowSize}(w_2, s)) \wedge \\ \dots \text{GetY}(\text{WindowPosition}(w_2, s)) < \text{GetY}(\text{WindowPosition}(w_1, s)) + \text{GetHeight}(\text{WindowSize}(w_1, s)) \end{aligned}$$

Front-to-Back Ordering:

$$\begin{aligned} \text{InFront}(w_1, w_2, s) \leftrightarrow \\ \dots \text{WindowHasState}(w_1, \text{DISPLAYED}, s) \wedge \\ \dots \text{WindowHasState}(w_2, \text{DISPLAYED}, s) \wedge \\ \dots \text{WindowOrder}(w_1, s) < \text{WindowOrder}(w_2, s) \end{aligned}$$

3.5 Initial State Axioms

```
ActiveWindow( $S_0$ ) = NIL
```

```
 $\forall w. \neg \text{Exists}(w, S_0)$ 
```

```
 $\forall w. \text{WindowHasState}(w, \text{NONEXISTENT}, S_0)$ 
```

4. Example Usage

To demonstrate the system, consider creating and manipulating windows:

```
% Create window W1 at position (100, 100) with size 400x300
```

```
Do(CreateWindow(W1, 100, 100, 400, 300),  $S_0$ )
```

```
% Activate the window
```

```
Do(ActivateWindow(W1), Do(CreateWindow(W1, 100, 100, 400, 300),  $S_0$ ))
```

```
% Create another window W2
```

```
Do(CreateWindow(W2, 200, 150, 300, 200),
```

```
... Do(ActivateWindow(W1),
```

```
..... Do(CreateWindow(W1, 100, 100, 400, 300),  $S_0$ )))
```

```
% Bring W1 to front
```

```
Do(BringToFront(W1), ...)
```

This representational system provides a complete formal framework for reasoning about window management operations in a GUI environment using situation calculus.